Baseball Strategy Agent

Reinforcement Learning Midterm Project

Ospanov Dayan | Student ID: 23B030308

1. Problem and motivation

The agent chooses whether to take or swing at each pitch in a simplified at-bat. Its objective is to maximize the probability of reaching base through a hit or a walk. A decision affects both the immediate outcome and the count faced next, so this is a sequential decision problem suitable for reinforcement learning.

SARSA learns action values from simulated experience. A non-RL alternative is a fixed rule, such as taking until two strikes and then swinging. Since the transition probabilities are known in this synthetic environment, dynamic programming would also be possible. The purpose here is to study learning from sampled transitions, rather than to model professional baseball accurately.

2. Environment and Markov assumption

A state is (balls, strikes), with balls in {0,1,2,3} and strikes in {0,1,2}: 12 nonterminal states and two actions, Take and Swing. Every episode starts at (0,0). A hit or fourth ball ends the at-bat with reward +1; an out or third strike ends it with -1. All other transitions give 0.

Action	Possible result	Probability	Effect
Take	Ball	0.40	Add one ball; four balls = walk
Take	Called strike	0.60	Add one strike; three strikes = out
Swing	Hit	0.24	Terminal success
Swing	Other out	0.18	Terminal failure
Swing	Foul	0.20	Add a strike only below two strikes
Swing	Miss	0.38	Add one strike; three strikes = out

Example: from (2,1), taking leads to (3,1) with probability 0.40 and reward 0. The count is Markov because the next-state distribution depends only on the current count and action under these fixed probabilities. Real baseball would require additional information about the batter, pitcher, pitch and history.

A 30-pitch safety limit truncates exceptionally long simulations. It is not counted as a baseball out, and the SARSA update still bootstraps at that limit. No truncations occurred in these experiments.

3. SARSA implementation

The notebook stores Q-values in a dictionary with two entries per state, initialized to zero. During training, epsilon-greedy selection chooses a random action with probability epsilon and otherwise chooses a highest-valued action. Training ties are broken randomly. SARSA uses the next action actually selected by this policy, making it an on-policy method [1].

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

Here r is the observed reward, alpha is the learning rate, and gamma discounts the next action value. The term in brackets is the temporal-difference error. At a true terminal state the target is just r; at a time-

limit truncation the next-state term is retained [2]. With gamma = 1, the return is the final +1 or -1 outcome when an episode terminates normally.

4. Experimental protocol

Each run trains for 10,000 at-bats with alpha = 0.1 and gamma = 1. The main epsilon is 0.20; sensitivity runs use 0.05 and 0.50. All settings use five seeds: 42, 123, 456, 789 and 2026. This gives 15 training runs. The episode budget is practical for 24 Q-values but does not guarantee convergence.

Final evaluation uses 10,000 fresh at-bats per seed and policy, with no Q updates and greedy SARSA actions; evaluation ties choose Take. Training transitions and actions use separate random streams, and evaluation uses different seed offsets. Random chooses either action equally, the fixed rule takes until two strikes, and always-swing swings at every count. The notebook contains the complete seeds and offsets.

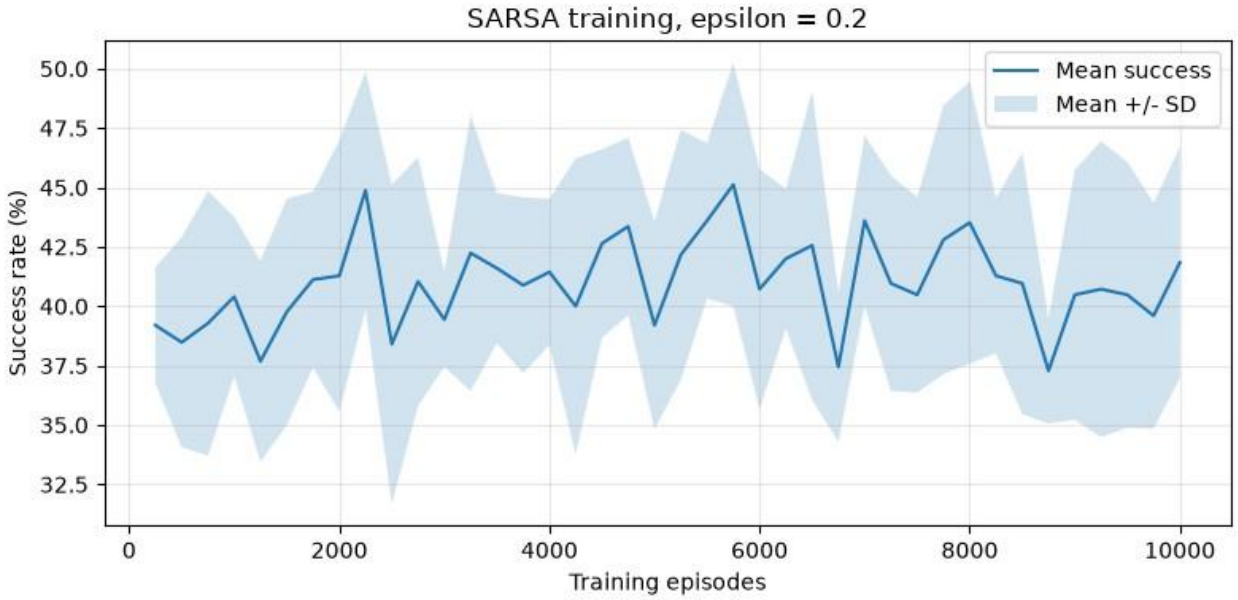


Figure 1. Training success rates in 250-episode windows for the main epsilon. The line is the mean across five seeds and shading shows one sample standard deviation. These rates include exploratory actions and are different from final greedy evaluation.

The training curve remains noisy. Constant exploration keeps trying alternative actions, and a constant learning rate lets later samples continue changing Q-values. The curve alone does not establish that the learned policy has converged.

5. Final evaluation

Policy	Success (%)	SD (pp)	Mean reward	Pitches
Random	36.28	0.28	-0.2744	2.928
Fixed rule	36.30	0.73	-0.2739	4.288
Always swing	47.47	0.51	-0.0506	2.013
SARSA eps=0.05	39.98	1.44	-0.2004	3.193
SARSA eps=0.2	40.87	6.05	-0.1826	3.094
SARSA eps=0.5	43.07	3.76	-0.1386	2.904

Each row summarizes five independent seed runs. SD is the sample standard deviation across their success rates, in percentage points; it is not a confidence interval. With no truncations, mean reward equals twice the success rate minus one.

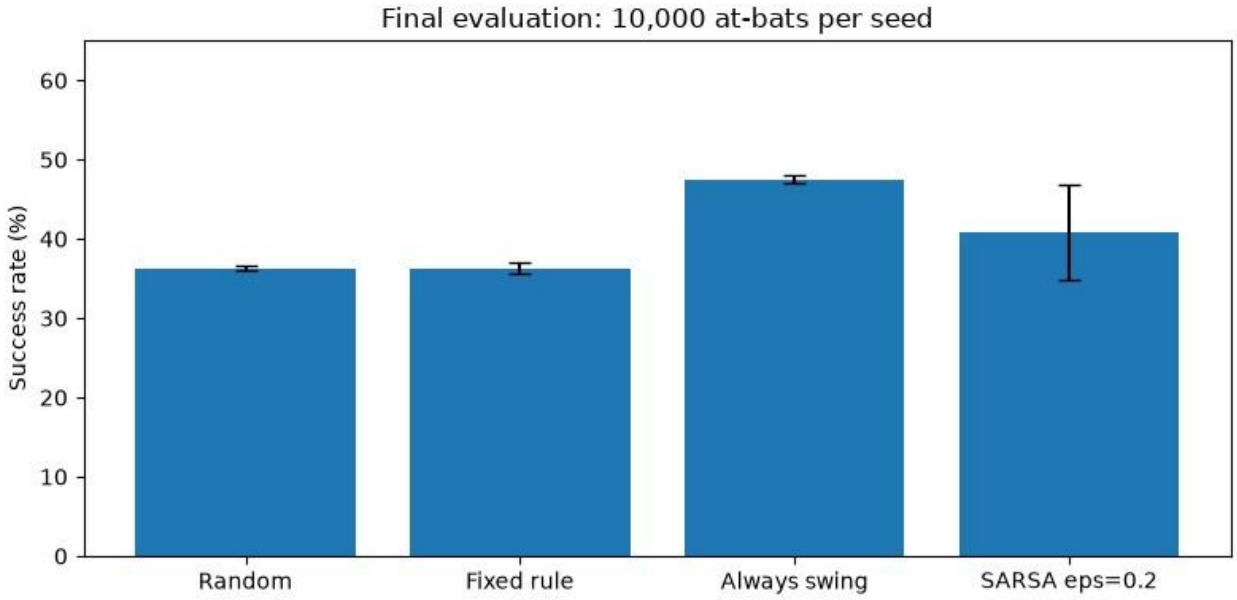


Figure 2. Main SARSA setting compared with three baselines; error bars show one sample standard deviation across seeds.

Main-setting SARSA succeeds in 40.87% of at-bats, compared with 36.28% for random and 36.30% for the fixed rule. It does not outperform always-swing, which reaches 47.47%. Its 6.05 percentage-point standard deviation also indicates substantial seed dependence. Five seeds and overlapping variation do not justify a claim of statistically significant improvement. The strong baseline is understandable from the model. Let v be always-swing success probability with k $_k$ strikes. Then $v_2 = 0.24 + 0.20v_2 = 0.30$; $v_1 = 0.24 + 0.58(0.30) = 0.414$; and $v_0 = 0.24 + 0.58(0.414) = 0.48012$. Its exact 48.012% success rate is close to the sampled result. This calculation evaluates that baseline; it is not a proof that SARSA learned an optimal policy.

6. Exploration sensitivity and learned decisions

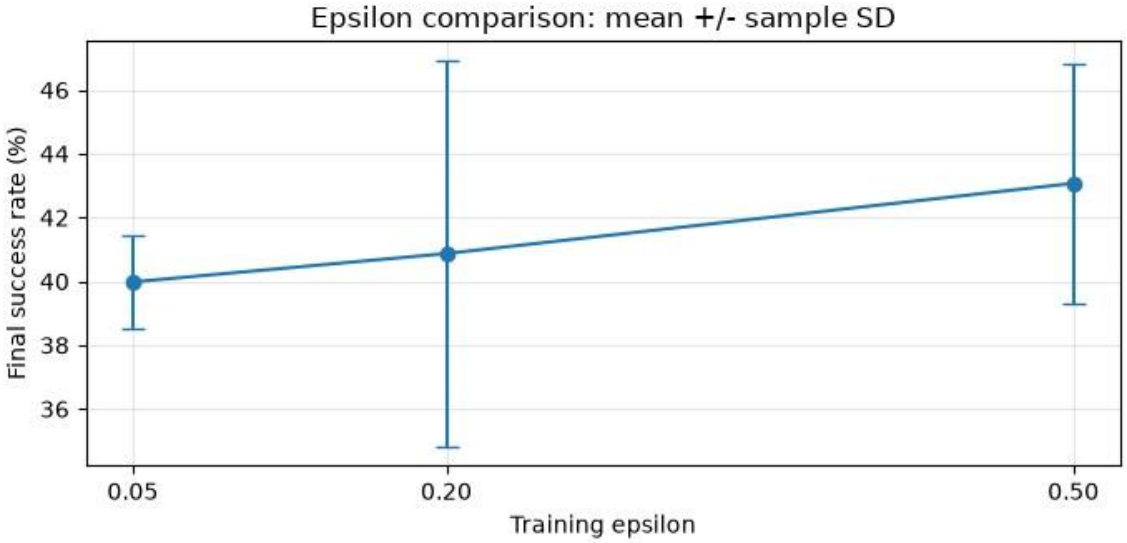


Figure 3. Final greedy success versus training epsilon. Points are five-seed means; error bars are sample standard deviations.

Epsilon 0.50 gives the largest observed SARSA mean (43.07%), followed by 0.20 (40.87%) and 0.05 (39.98%). More exploration may improve coverage, but the variation overlaps and this small comparison does not establish a universally best epsilon. At epsilon 0.20, seed 2026 obtains only 30.28%, while the other seeds range from 41.40% to 44.73%.

Balls	0 strikes		1 strike	2 strikes
0	Swing	Take	Swing	
1	Take Swing	Swing		
2	Take Swing	Swing		
3	Take Take	Swing		

Table: greedy decisions for seed 42 at epsilon 0.20. This is one run, not a consensus policy. Its mix of Take and Swing shows that learning does not simply reproduce the strongest baseline.

7. Limitations and conclusion

SARSA learns through interaction without consulting the transition probabilities in its update. It improves the observed mean over two weaker baselines but falls short of always-swing. Constant alpha and epsilon are plausible contributors to unstable estimates, not experimentally proven causes. Decaying schedules and more seeds would be useful follow-up experiments.

The environment uses synthetic probabilities shared by every count and ignores pitch location, player differences and game context. It is an educational at-bat model rather than a practical baseball decision system. Keeping the model unchanged, even when the agent loses to a simple baseline, makes the comparison informative.

AI assistance and authorship

I completed the experiments, analysis, report and learning reflection. OpenAI Codex helped with the Python implementation, testing and debugging, and provided initial project drafts. I reviewed and revised the material and take responsibility for the final work.

References

[1] Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, second edition, Section 6.4 (SARSA). https://incompleteideas.net/book/RLbook2018.pdf

[2] Farama Foundation. *Handling Time Limits*, Gymnasium documentation. gymnasium.farama.org/tutorials/gymnasium_basics/handling_time_limits/

[3] CSCI3235. *Midterm 1 Take-Home Project*. Supplied assignment handout.

Reproducing the results

Open baseball_agent.ipynb, select a Python environment containing NumPy, Matplotlib and ipykernel, and run all cells from top to bottom. The notebook defines the environment, trains all agents, evaluates all policies and displays the tables and three plots. It requires no dataset or project-specific Python modules. README.md contains the environment instructions.

Learning record

Ospanov Dayan | 23B030308

Personal reflection

Through this project, I developed a clearer understanding of the SARSA implementation process and how to formulate a real-world-inspired problem as a Markov decision process. Defining the states, actions, transition probabilities, and rewards helped me connect the theoretical components of reinforcement learning with their practical implementation.

Developing and evaluating the agent also gave me more realistic expectations about reinforcement learning projects. Implementing an algorithm is only one part of the process; the assumptions of the environment and the evaluation procedure strongly influence how results should be interpreted. Although AI tools can assist with implementation, their output requires careful validation and an understanding of the underlying algorithm.

I selected the policies used for experimental comparison, including the always-swing strategy. This simple policy achieved stronger results than SARSA in the simulated environment, demonstrating the importance of comparing learned agents with meaningful baselines. However, this finding applies to the model's simplified assumptions and should not be generalized directly to real baseball. An area I would like to understand better is how to design transition probabilities and rewards that represent realistic situations while keeping the model manageable.

Learning evidence

Item	Record
Implementation evidence	The notebook defines the MDP transitions and applies the SARSA update during training.
Experimental contribution	Selection of the policies compared in the experiments, including always-swing. The saved evaluation table and plots document their results.
Idea that became clearer	Connecting states, actions, transition probabilities and rewards to a working reinforcement learning environment.
Further learning goal	Designing realistic transition probabilities and rewards while maintaining a manageable model.

Code verification

I added and executed a deterministic check of the two-strike foul rule in the notebook. Starting at count (0,2), the check selected Swing and supplied a random draw of 0.50, which selects the foul branch. The observed result was ((0,2), 0, False): the count remained unchanged, the reward was zero, and the episode continued. An assertion comparing the result with the expected tuple passed.